



CRIME INVESTIGATION SYSTEM

Plano de Desenvolvimento Técnico

Versão 1.0 • Sistema Web com Multi-Agent IA

1. Visão Geral do Projeto

O Crime Investigation System é um jogo investigativo web onde o usuário assume o papel de detetive para resolver crimes. O diferencial central é que cada personagem com quem o investigador interage é um agente de IA independente, com personalidade, conhecimento e segredos próprios. Ao final, o jogador submete sua teoria e recebe uma avaliação baseada na solução real do caso.

Características Principais

- ▶ Personagens como Agentes IA com system prompts individuais
- ▶ Descoberta progressiva — personagens e locais desbloqueados por gatilhos
- ▶ Interrogatório em grupo com dinâmica entre personagens
- ▶ Locais investigáveis com pistas colecionáveis
- ▶ Modo Easy com parceiro detetive (também um agente IA)
- ▶ Avaliação final com porcentagem de acerto por dimensão do crime

2. Stack Tecnológica

Todas as tecnologias são compatíveis com VS Code e possuem excelente suporte via extensões e IntelliSense.

Camada	Tecnologia	Justificativa
Frontend	React + TypeScript + Vite	Componentização, tipagem forte, hot reload instantâneo — ideal pra UI complexa de chat + mapa
Estilização	Tailwind CSS	Utilitário, sem conflitos de escopo, fácil customização do tema noir do

Camada	Tecnologia	Justificativa
		jogo
Estado Global	Zustand	Leve, sem boilerplate. Guarda game state, pistas, salas ativas e histórico
Comunicação RT	Socket.io (WebSocket)	Essencial para chat em grupo e respostas assíncronas dos agentes em tempo real
Backend	Node.js + Express + TypeScript	Mesmo ecossistema do frontend, tipagem compartilhável (DTOs)
Banco de Dados	PostgreSQL + Prisma ORM	Prisma facilita migrações e queries tipadas no VS Code
IA	Anthropic Claude API	Melhor custo-benefício para múltiplos agentes com system prompts longos
Cache/Sessão	Redis	Guarda game state em memória, histórico de conversas e locks de sala
Containers	Docker + docker-compose	Sobe tudo (API, DB, Redis) com um comando no terminal do VS Code

3. Estrutura de Pastas no VS Code

Monorepo simples com separação clara de responsabilidades:

```

crime-system/
├── apps/
│   ├── frontend/           # React + Vite
│   │   ├── src/
│   │   │   ├── components/ # UI components
│   │   │   │   ├── Map/    # Mapa do caso (nós + grafos)
│   │   │   │   ├── Chat/   # Sala de interrogatório
│   │   │   │   ├── Scene/  # Locais inspecionáveis
│   │   │   │   ├── Notebook/ # Caderno do detetive
│   │   │   │   └── Companion/ # Parceiro IA (modo easy)
│   │   │   ├── store/      # Zustand slices
│   │   │   ├── hooks/      # Custom hooks (useSocket, useCase...)
│   │   │   └── types/       # TypeScript interfaces compartilhadas
│   │   └── vite.config.ts
│   └── backend/            # Node + Express
│       ├── src/
│       │   ├── agents/     # Lógica dos agentes IA
│       │   │   ├── AgentOrchestrator.ts
│       │   │   ├── CharacterAgent.ts
│       │   │   ├── LocationAgent.ts
│       │   │   └── JudgeAgent.ts
│       │   └── game/        # Game State Manager
│       │       ├── GameSession.ts
│       │       ├── DiscoveryGraph.ts
│       │       └── TriggerEngine.ts

```

```

├── cases/           # Casos (JSON + loader)
├── routes/          # Endpoints REST
├── sockets/         # Handlers WebSocket
├── db/              # Prisma client + migrations
├── prisma/schema.prisma
├── packages/
├──   ├── shared-types/  # DTOs e interfaces compartilhadas
├── docker-compose.yml
├── .vscode/
├──   ├── extensions.json  # Extensões recomendadas
├──   └── launch.json      # Debug configs (frontend + backend)

```

4. Plano de Fases

O desenvolvimento segue uma progressão onde cada fase gera um produto jogável e testável.

Fase	Título	Descrição	Tempo Est.	Dependências
Fase 1	Setup & Fundação	Monorepo, Docker, Prisma, conexão com Claude API, estrutura base de pastas e VS Code configs	3-4 dias	—
Fase 2	Motor de Casos	Schema JSON dos casos, loader, game state no Redis, endpoints REST básicos	4-5 dias	Fase 1
Fase 3	Chat 1-a-1	Agente por personagem, system prompt dinâmico, histórico de conversa, UI de interrogatório	5-6 dias	Fase 2
Fase 4	Locais & Pistas	LocationAgent, cena inspecionável, pistas colecionáveis, caderno do detetive	4-5 dias	Fase 3
Fase 5	Discovery Graph	TriggerEngine, nós bloqueados/desbloqueados, mapa interativo com grafos	5-7 dias	Fase 4
Fase 6	Chat em Grupo	Multi-agente em paralelo, sequenciamento de respostas, dinâmica entre personagens	6-8 dias	Fase 5
Fase 7	Parceiro Detetive	CompanionAgent com acesso ao game state, modo easy, integração no interrogatório	4-5 dias	Fase 6
Fase 8	Avaliação Final	JudgeAgent com doc mestre, avaliação multi-dimensão, tela de resultado com %, feedback	4-5 dias	Fase 7
Fase 9	Polimento & Deploy	Autenticação, ranking, múltiplos casos, deploy (Railway/Render), testes E2E	7-10 dias	Fase 8

5. Detalhamento — Fase 1 (Setup)

Ponto de partida. Ao final desta fase, o ambiente estará 100% funcional e o time pode começar a desenvolver features sem atrito.

5.1 Extensões VS Code (extensions.json)

- ▶ ESLint + Prettier — formatação automática
- ▶ Prisma — syntax highlight e autocompletar no schema
- ▶ Tailwind CSS IntelliSense — autocomplete de classes
- ▶ Thunder Client — testes de API sem sair do VS Code
- ▶ Docker — gerenciar containers inline
- ▶ GitLens — histórico e diff avançados
- ▶ Error Lens — erros inline no editor

5.2 Comandos de Setup

No terminal integrado do VS Code (Ctrl+`):

```
# 1. Clonar e instalar dependências
npm create vite@latest frontend -- --template react-ts
cd backend && npm init -y && npm i express socket.io prisma

# 2. Subir infraestrutura
docker-compose up -d # PostgreSQL + Redis

# 3. Configurar banco
npx prisma migrate dev --name init

# 4. Variáveis de ambiente (.env)
ANTHROPIC_API_KEY=sk-ant-...
DATABASE_URL=postgresql://user:pass@localhost:5432/crimedb
REDIS_URL=redis://localhost:6379
```

5.3 launch.json (Debug)

Configurar dois launchers para rodar frontend e backend simultaneamente com F5:

```
{
  "configurations": [
    {
      "name": "Backend",
      "type": "node",
      "request": "launch",
      "program": "${workspaceFolder}/apps/backend/src/index.ts",
      "runtimeArgs": ["-r", "ts-node/register"]
    },
    {
      "name": "Frontend",
      "type": "chrome",
      "request": "launch",
      "url": "http://localhost:5173"
    }
  ]
}
```

6. Modelagem de Dados (Prisma)

Schema principal. Cada caso é autocontido em JSON — o banco guarda progresso e sessões.

```
model Case {
  id          String    @id @default(cuid())
  title       String
  difficulty   String    // easy | normal | hard
  masterDoc   Json       // solução completa, invisível ao jogador
  nodes       Json       // personagens, locais, triggers
  sessions    Session[]
}

model Session {
  id          String    @id @default(cuid())
  caseId      String
  case        Case      @relation(fields: [caseId], references: [id])
  unlockedNodes Json    // array de node IDs desbloqueados
  clues       Json       // pistas coletadas
  notebook    String     // anotações livres do jogador
  mode        String     // easy | normal
  finalTheory  String?   // teoria submetida
  score       Float?     // porcentagem de acerto
  createdAt   DateTime @default(now())
}
```

7. Arquitetura dos Agentes IA

7.1 System Prompt de Personagem

Cada personagem recebe um system prompt gerado dinamicamente a partir do JSON do caso:

```
Você é [NOME], [DESCRIÇÃO].

PERSONALIDADE: [traços de personalidade]
O QUE VOCÊ SABE: [lista de fatos que este personagem conhece]
O QUE VOCÊ ESCONDE: [segredos – nunca revele diretamente]
COMO SE COMPORTAR: [nervoso, evasivo, cooperativo, mentiroso...]

REGRAS ABSOLUTAS:
- Nunca quebre o personagem
- Se pressionado sobre [X], desvie com [comportamento Y]
- Você pode mencionar [NOME_PERSONAGEM_B] se falar sobre [ASSUNTO]
```

7.2 Orquestrador de Grupo

No chat em grupo, as respostas são encadeadas para criar dinâmica real entre personagens:

```
// AgentOrchestrator.ts - simplificado
async function handleGroupMessage(roomId, message, participants) {
  const responses = [];
  for (const agentId of participants) {
    const history = await getHistory(roomId, agentId);
    // Injeta respostas anteriores no contexto
    const ctx = [...history, ...responses.map(toAssistantMsg)];
    const reply = await callClaude(agentId, ctx, message);
    responses.push({ agent: agentId, text: reply });
    // Emite via WebSocket em tempo real
    io.to(roomId).emit('agent_reply', { agentId, text: reply });
  }
}
```

7.3 Agente Juiz (Avaliação Final)

Ao submeter a teoria, o JudgeAgent compara com o documento mestre e pontua por dimensão:

```
Você é um juiz imparcial avaliando a teoria de um detetive.

SOLUÇÃO REAL: [documento mestre completo]
TEORIA DO JOGADOR: [texto livre submetido]

Avalie cada dimensão de 0 a 100 e retorne JSON:
{
  "suspect": { "score": 85, "feedback": "..." },
  "motive": { "score": 70, "feedback": "..." },
  "method": { "score": 90, "feedback": "..." },
  "location": { "score": 100, "feedback": "..." },
  "time": { "score": 60, "feedback": "..." },
  "total": 82,
  "narrative": "Resumo da solução real..."
}
```

8. Estrutura JSON de um Caso

Exemplo do caso de teste para o MVP — 'A Morte na Mansão Blackwood':

```
{
  "id": "caso_001",
  "title": "A Morte na Mansão Blackwood",
  "difficulty": "normal",
  "masterDoc": {
    "suspect": "Carlos (mordomo)",
    "motive": "herança negada no testamento",
    "method": "veneno no vinho do jantar",
    "time": "22:30",
    "location": "Sala de Jantar"
  }
}
```

```

},
"nodes": {
  "characters": [
    { "id": "carlos", "name": "Carlos",
      "role": "Mordomo", "unlocked": true,
      "personality": "nervoso e evasivo",
      "knows": ["jantar às 20h", "Helena discutiu com Lord B."],
      "hides": ["saiu às 22h", "leu o testamento"] },
    { "id": "helen", "name": "Helena",
      "unlocked": false,
      "unlockTrigger": "talked_to:carlos + keyword:Helena" }
  ],
  "locations": [
    { "id": "sala_jantar", "name": "Sala de Jantar",
      "unlocked": true,
      "elements": [
        { "id": "taca", "label": "Taça de vinho",
          "clue": true, "clueId": "residuo_veneno" },
        { "id": "cadeira", "label": "Cadeira virada",
          "clue": false, "description": "Parece ter caído na pressa" }
      ] }
  ]
}

```

9. Checklist por Fase

Fase 1 — Setup

- ▶ ☐ Monorepo criado com apps/frontend e apps/backend
- ▶ ☐ docker-compose.yml com PostgreSQL e Redis
- ▶ ☐ .env configurado com ANTHROPIC_API_KEY
- ▶ ☐ Prisma schema criado e primeira migration rodada
- ▶ ☐ VS Code extensions.json e launch.json configurados
- ▶ ☐ Teste de chamada à Claude API funcionando

Fase 2 — Motor de Casos

- ▶ ☐ Schema JSON de caso definido e validado
- ▶ ☐ CaseLoader lendo casos da pasta /cases
- ▶ ☐ GameSession criando sessão no Redis
- ▶ ☐ Endpoints: POST /sessions, GET /sessions/:id

Fase 3 — Chat 1-a-1

- ▶ ☐ CharacterAgent com system prompt dinâmico

- ▶ ☐ Histórico de conversa mantido por sessão+agente
- ▶ ☐ WebSocket emitindo respostas em streaming
- ▶ ☐ UI de chat com balões de diálogo e typing indicator

Fase 4 — Locais & Pistas

- ▶ ☐ LocationAgent com elementos inspecionáveis
- ▶ ☐ Clue system: coletar, listar, deletar do caderno
- ▶ ☐ UI de cena com elementos clicáveis
- ▶ ☐ Caderno do detetive com anotações livres

Fase 5 — Discovery Graph

- ▶ ☐ TriggerEngine monitorando keywords e ações
- ▶ ☐ Nós bloqueados com visual diferenciado no mapa
- ▶ ☐ Notificação ao jogador quando novo nó é desbloqueado
- ▶ ☐ Mapa interativo com React Flow ou D3

Fase 6 — Chat em Grupo

- ▶ ☐ AgentOrchestrator encadeando respostas
- ▶ ☐ UI de sala com múltiplos avatares
- ▶ ☐ Contexto cruzado entre agentes na mesma sala
- ▶ ☐ Gatilhos de desbloqueio via conversa em grupo

Fase 7 — Parceiro Detetive

- ▶ ☐ CompanionAgent com acesso ao game state completo
- ▶ ☐ Lógica de sugestão baseada em pistas não exploradas
- ▶ ☐ Integração do parceiro nas salas de interrogatório
- ▶ ☐ Toggle modo easy/normal na tela de seleção de caso

Fase 8 — Avaliação Final

- ▶ ☐ JudgeAgent recebendo teoria + documento mestre
- ▶ ☐ Retorno JSON com score por dimensão
- ▶ ☐ Tela de resultado com breakdown visual do acerto
- ▶ ☐ Revelação narrativa da solução real

Fase 9 — Polimento

- ▶ ☐ Autenticação (NextAuth ou Clerk)
- ▶ ☐ Ranking global por caso
- ▶ ☐ 2+ casos completos
- ▶ ☐ Testes E2E com Playwright

- ▶ [] Deploy na Railway ou Render

10. Próximos Passos Imediatos

Para começar ainda hoje no VS Code:

1. Criar a pasta do projeto e abrir no VS Code
2. Copiar o docker-compose.yml e subir PostgreSQL + Redis
3. Configurar o Prisma e rodar a primeira migration
4. Escrever o JSON do caso de teste (Mansão Blackwood)
5. Fazer a primeira chamada à Claude API com um personagem simples e validar o retorno



Dica: Comece pelo menor loop jogável possível — um personagem respondendo uma pergunta via API. Isso valida toda a integração antes de construir a UI.